CADERNO DE ENGENHARIA / 03

Conciliação de pagamentos

Do enunciado ao código.

Uma solução pequena, explicada por dentro.
Java 17, estruturas de dados, testes e raciocínio.
Para construir, entender e conseguir explicar.

UM LOTE. QUATRO RESPOSTAS.

`pag-2`	**CONCILIADO**
`pag-1`	**VALOR_DIVERGENTE**
`pag-3`	**NAO_LIQUIDADO**
`pag-4`	**NAO_PREVISTO**

EDIÇÃO 29.10.2026 | WESLEY VIEIRA

Exercício educacional fictício. Não é uma questão oficial do Nubank.

Seu mapa de leitura

Leia em sequência para construir a solução. Depois, volte aos capítulos pelo sumário ou pelos marcadores do PDF. As páginas de código e as explicações foram geradas a partir dos arquivos reais deste repositório.

Do enunciado à implementação, uma decisão de cada vez 3

1. Transforme o enunciado em um contrato 3

2. Pense na busca antes de escrever os laços 5

3. O Java necessário para ler esta solução 6

4. Modele os dados sem antecipar responsabilidades 8

5. Leia o serviço pelo fluxo principal 11

6. Percorra o lote do enunciado, sem pular etapas 18

7. Validar o lote inteiro não é juntar todos os erros 20

8. Proteja as entradas e a resposta 21

9. Explique a complexidade com precisão 22

10. Use TDD como um roteiro de construção 22

11. Entenda o que os testes realmente verificam 24

12. Maven: como o projeto é montado 25

13. O exemplo executável e a fronteira da aplicação 30

14. Resolva erros sem alterar o contrato 33

15. Exercícios para confirmar o entendimento 34

16. Explique a solução com suas próprias palavras 36

Referências para continuar estudando 37

Do enunciado à implementação, uma decisão de cada vez

Um guia de estudo para o Desafio #03 · Java 17 · JUnit 5

Conciliar é comparar duas visões do mesmo pagamento e explicar o que aconteceu. Neste desafio, uma lista informa o que era esperado; outra informa o que o parceiro efetivamente liquidou. Sua tarefa é relacionar referências, comparar valores e devolver uma classificação previsível.

Você não precisa conhecer um framework para acompanhar este livro. Vamos construir o raciocínio com listas, mapas, condições e pequenos tipos de dados. Quando aparecer uma palavra nova, ela será conectada à linha de código que precisa dela.

O exercício é fictício, inspirado no contexto de um banco digital. Ele não é uma questão oficial do Nubank. A referência identifica um pagamento, não uma fatura inteira: somar pagamentos de uma fatura, atualizar saldos e chamar sistemas externos estão fora do contrato.

Leia primeiro para entender o fluxo. Depois execute o projeto e refaça os exemplos com papel e lápis. A explicação linha a linha serve como apoio à leitura do código; não é necessário decorar cada palavra para começar.

1. Transforme o enunciado em um contrato

Um contrato responde três perguntas: o que entra, o que sai e o que torna uma entrada inválida. Separar essas perguntas evita começar pela ferramenta e descobrir tarde que a solução atende a uma regra diferente.

A entrada contém duas listas de **Lancamento**. Cada lançamento possui uma referência textual e um valor inteiro em centavos. A saída é uma lista de **Resultado**, formada por referência e status. Portanto, não precisamos acrescentar à resposta o valor esperado, o valor liquidado ou uma mensagem livre: esses campos não foram pedidos.

Existe exatamente um resultado para cada referência que aparece em pelo menos uma das listas. Se uma referência está nas duas, ela continua produzindo apenas um resultado. Se as duas listas estão vazias, não existe referência para classificar, e a saída também fica vazia.

Os quatro estados possíveis

Situação da referência	Esperado	Liquidado	Status
Presente nos dois lados, valores iguais	2.000 centavos	2.000 centavos	CONCILIADO
Presente nos dois lados, valores diferentes	1.000 centavos	900 centavos	VALOR_DIVERGENTE
Presente somente nos esperados	3.000 centavos	Ausente	NAO_LIQUIDADO
Presente somente nos liquidados	Ausente	4.000 centavos	NAO_PREVISTO

“Ausente” não significa zero. Zero é um valor inválido neste exercício; ausência significa que aquela chave não existe na lista correspondente. Preservar essa diferença será importante quando o código utilizar **Long** para representar uma consulta que pode não encontrar nada.

A ordem também é parte do contrato. Primeiro aparecem todas as referências dos esperados, na sequência original. Depois aparecem somente as referências exclusivas dos liquidados, na ordem de chegada nessa segunda lista. Não é uma ordenação alfabética nem uma organização por status.

O que deve ser rejeitado

Uma lista nula é inválida, mas uma lista vazia é válida. Um elemento nulo dentro da lista também é inválido. A referência não pode ser nula, vazia ou composta apenas de caracteres considerados espaços em branco por **String.isBlank()**. O valor deve ser estritamente positivo.

Uma mesma referência não pode aparecer duas vezes dentro de uma lista, mesmo que os valores sejam iguais. Entretanto, aparecer uma vez em cada lista é permitido: essa é justamente a situação que possibilita comparar os pagamentos.

A comparação da referência é exata. **pag-1**, **PAG-1** e `" pag-1 "` são três chaves diferentes. Os espaços em uma referência não vazia não são removidos automaticamente. Não use **trim()**, conversão para minúsculas ou outra normalização “para ajudar”: isso mudaria a regra recebida.

A documentação de String no Java 17 detalha as operações de texto. Aqui, fazemos a escolha de negócio de validar conteúdo em branco sem transformar as referências aceitas.

2. Pense na busca antes de escrever os laços

Uma primeira ideia costuma ser: para cada esperado, percorrer todos os liquidados até encontrar a mesma referência. A ideia é simples de visualizar, mas pode repetir muito trabalho. Com mil esperados e mil liquidados, uma busca completa para cada esperado pode fazer aproximadamente um milhão de comparações.

O enunciado pede tempo médio proporcional à soma dos tamanhos das listas. Precisamos organizar os dados para encontrar rapidamente um valor por referência, em vez de procurar sempre desde o início.

Um **Map** associa uma chave a um valor. Neste problema, a chave é a referência do pagamento e o valor é a quantidade de centavos. Podemos pensar no índice abaixo:

Chave	Valor
pag-2	2000
pag-1	1000
pag-3	3000

A pergunta “qual o valor de pag-1?” passa a ser uma consulta por chave. O mapa também ajuda a detectar duplicatas durante sua construção.

Por que LinkedHashMap?

O tipo **Map** descreve as operações de um mapa, mas não promete uma ordem universal de percurso. **HashMap** não oferece a ordem de inserção exigida aqui. **TreeMap** ordenaria as chaves, o que também não corresponde à sequência recebida.

A implementação escolhida é **LinkedHashMap**, na configuração padrão de ordem de inserção. Ela permite buscar por chave e percorrer as entradas na sequência em que foram incluídas. Essa propriedade vale tanto para o índice dos esperados quanto para o dos liquidados. Consulte a documentação de LinkedHashMap para distinguir ordem de inserção de configurações baseadas em acesso.

O plano fica pequeno o bastante para ser explicado antes do código:

1. Validar e indexar os esperados.
2. Validar e indexar os liquidados.
3. Percorrer os esperados, produzindo seus resultados.
4. Percorrer os liquidados e acrescentar apenas as referências ausentes dos esperados.

5. Devolver uma lista não modificável.

Há dois mapas porque a duplicidade tem escopo por lista. Misturar tudo em um único mapa de entrada dificultaria distinguir a repetição inválida dentro de um lado da presença válida nos dois lados.

3. O Java necessário para ler esta solução

Package, arquivo e import

A declaração **package br.com.wesley.conciliacao;** identifica o pacote dos tipos. No projeto, os arquivos de produção ficam em **src/main/java/br/com/wesley/conciliacao**. Essa organização ajuda o compilador e as ferramentas a localizar classes sem confundir nomes iguais de partes diferentes de um sistema.

Um **import** permite usar o nome curto de um tipo de outro pacote. **import java.util.List;** autoriza escrever **List** em vez de repetir **java.util.List**. Ele não instala uma dependência, não copia o código da biblioteca e não cria um objeto.

As coleções usadas pertencem à biblioteca padrão do Java. Não é necessário instalar um pacote separado para obter **ArrayList**, **LinkedHashMap**, **List** ou **Map**. Já o JUnit é uma dependência de testes declarada no Maven.

Os cinco tipos da aplicação estão no mesmo pacote, por isso se enxergam sem imports entre eles. **String**, **Long**, **System** e **IllegalArgumentException** pertencem a **java.lang**, pacote disponibilizado automaticamente. Não ver um import desses nomes é normal.

Classe, objeto e método

Conciliador é uma classe: ela descreve o comportamento do serviço. **new Conciliador()** cria uma instância dessa classe. O método **conciliar** recebe argumentos e devolve uma lista de resultados.

public permite que o tipo ou método seja acessado de fora de seu pacote, respeitadas as demais regras da linguagem. **private** limita os métodos auxiliares à própria classe. Essa separação deixa um ponto de entrada claro e esconde detalhes que não precisam virar contrato público.

A classe é **final**, portanto não pode ser estendida por herança. Isso não torna automaticamente todos os seus objetos imutáveis. A independência entre chamadas vem do fato de os dados de trabalho serem variáveis locais, sem campos mutáveis compartilhados.

O método público não precisa declarar **throws IllegalArgumentException** para lançar essa exceção. Ela é uma exceção não verificada: o chamador pode capturá-la, mas o compilador não obriga uma cláusula de tratamento.

Generics: o que cabe em cada coleção

Em `List<Lancamento>`, o trecho entre os sinais de menor e maior informa que a lista contém lançamentos. Em `List<Resultado>`, ela contém resultados. O compilador utiliza essa informação para detectar várias misturas indevidas antes da execução.

`Map<String, Long>` possui dois parâmetros: o tipo da chave e o tipo do valor. `Map.Entry<String, Long>` representa um par encontrado durante o percurso do mapa. A entrada fornece a chave com **getKey()** e o valor com **getValue()**.

O diamante vazio em `new ArrayList<>()` não significa “sem tipo”. O compilador infere o parâmetro a partir do contexto. A variável pode ser declarada como `List<Resultado>` e receber uma instância de **ArrayList**, porque essa classe implementa a interface **List**.

long e Long: número e objeto

long é um tipo primitivo inteiro de 64 bits com sinal. **Long** é o tipo objeto que representa esse valor. Um **long** não pode receber **null**; uma variável **Long** pode.

Os valores dos lançamentos são **long**, como solicitado. O mapa usa **Long** porque parâmetros genéricos não aceitam tipos primitivos. Quando o valor entra no mapa, o Java pode convertê-lo automaticamente de **long** para **Long**: isso se chama boxing. A conversão inversa é unboxing.

Ao consultar uma chave inexistente com **get**, recebemos **null**. Isso explica a assinatura de **classificar**: o esperado é **long**, pois existe naquele percurso; o liquidado é **Long**, pois pode estar ausente.

A solução verifica primeiro se o liquidado é nulo. Só depois utiliza **longValue()** para comparar números. Sem essa guarda, tentar extrair o primitivo de uma referência nula causaria **NullPointerException**.

Evite comparar dois objetos **Long** com **==** esperando sempre comparar valores. Entre referências, esse operador testa identidade de objeto. Alguns valores podem compartilhar objetos e mascarar o erro em exemplos pequenos. Na solução, a comparação acontece entre primitivos, deixando a intenção explícita. A distinção está documentada na API de Long.

Centavos, L e separadores visuais

2_000L representa dois mil centavos, equivalentes a R$ 20,00. O sublinhado facilita a leitura do literal e não altera seu valor. O sufixo **L** indica um literal do tipo **long**.

Usar inteiros em centavos evita representar estes valores monetários como aproximações binárias de números com casas decimais. O exercício compara os valores recebidos; não calcula juros, não converte moedas e não soma uma fatura.

Isso também importa para valores grandes. A solução não converte os centavos para **int**, que possui alcance menor. Como não há soma nem multiplicação do dinheiro no algoritmo, comparar valores positivos até **Long.MAX_VALUE** não introduz overflow aritmético. Se o requisito passar a incluir totais, essa nova operação precisará de uma análise própria.

null e curto-circuito

Observe a condição **referencia == null || referencia.isBlank()**. O operador **||** avalia primeiro o lado esquerdo. Se ele já for verdadeiro, o lado direito não é executado.

Assim, quando a referência é nula, **isBlank()** não é chamado. Quando não é nula, a segunda parte verifica se falta conteúdo significativo. Inverter a ordem das verificações criaria o risco de chamar um método sobre **null**.

Não substitua **||** por **|** nesse trecho. Embora o operador simples também possa aparecer em expressões booleanas, ele não fornece o mesmo curto-circuito. A avaliação condicional é definida na especificação da linguagem Java 17.

4. Modele os dados sem antecipar responsabilidades

Lancamento: aquilo que chegou

O record **Lancamento** reúne a referência e os centavos. Um record fornece construtor, acessores e operações de igualdade associadas aos seus componentes, reduzindo o código repetitivo de um pequeno portador de dados.

Código completo: Lancamento.java

Os números à esquerda são referências de leitura; não fazem parte do arquivo Java ou XML.

```
1  package br.com.wesley.conciliacao;
2
3  public record Lancamento(String referencia, long valorCentavos) {
4  }
```

Cada linha, uma intenção

Linha	Instrução e explicação
1	`package br.com.wesley.conciliacao;` Coloca o tipo de entrada no pacote da solução, junto do conciliador.
3	`public record Lancamento(String referencia, long valorCentavos) {` Declara o record público com referência String e valor primitivo long; o Java fornece construtor, acessores e igualdade pelos componentes.
4	`}` Fecha o corpo vazio do record; não há validação no construtor, pois ela pertence ao serviço.

O acesso é feito por **referencia()** e **valorCentavos()**, não por getters obrigatoriamente chamados **getReferencia()** e **getValorCentavos()**. Os acessores têm os nomes dos componentes.

Este record é deliberadamente passivo. Ele pode ser construído com uma referência nula ou um valor negativo; quem rejeita esses dados, neste projeto, é o serviço de conciliação. Isso mantém todas as regras de aceitação do lote no mesmo lugar e permite que os testes entreguem dados inválidos ao ponto de entrada que precisa validá-los.

Não acrescente validação ao construtor do record sem reconsiderar o contrato e os testes. Uma exceção lançada durante a montagem da entrada ocorre antes de **conciliar** ser chamado. Seria uma mudança de responsabilidade, não apenas uma melhoria cosmética.

Status: quatro alternativas conhecidas

Um **enum** descreve um conjunto fechado de constantes com nome. Ele evita que cada parte do programa invente uma grafia diferente para a mesma classificação.

Código completo: Status.java

Os números à esquerda são referências de leitura; não fazem parte do arquivo Java ou XML.

```
1  package br.com.wesley.conciliacao;
2
3  public enum Status {
4      CONCILIADO,
5      VALOR_DIVERGENTE,
6      NAO_LIQUIDADO,
7      NAO_PREVISTO
8  }
```

Cada linha, uma intenção

Linha	Instrução e explicação
1	`package br.com.wesley.conciliacao;` Coloca o enum no pacote da solução para ser utilizado pelos resultados e pelo serviço.
3	`public enum Status {` Declara o enum público que restringe os status às constantes definidas a seguir.
4	`CONCILIADO,` Define CONCILIADO para uma referência presente nos dois lados com valores iguais.
5	`VALOR_DIVERGENTE,` Define VALOR_DIVERGENTE para uma referência presente nos dois lados com valores diferentes.

Linha	Instrução e explicação
6	`NAO_LIQUIDADO,` Define NAO_LIQUIDADO para uma referência encontrada somente nos esperados.
7	`NAO_PREVISTO` Define NAO_PREVISTO para uma referência encontrada somente nos liquidados; é a última constante.
8	`}` Fecha a declaração do enum.

Usar **Status.NAO_LIQUIDADO** é diferente de guardar uma String livre como “nao liquidado”. O compilador consegue verificar que o argumento pertence ao tipo **Status**. Os nomes também aparecem de maneira consistente nos testes e na saída do exemplo.

Resultado: uma decisão por referência

O record **Resultado** contém a referência classificada e o status correspondente.

Código completo: Resultado.java

Os números à esquerda são referências de leitura; não fazem parte do arquivo Java ou XML.

```
1  package br.com.wesley.conciliacao;
2
3  public record Resultado(String referencia, Status status) {
4  }
```

Cada linha, uma intenção

Linha	Instrução e explicação
1	`package br.com.wesley.conciliacao;` Declara o mesmo pacote dos demais tipos de domínio.
3	`public record Resultado(String referencia, Status status) {` Declara o record público de saída com referência e Status; seus acessores são referencia() e status().
4	`}` Fecha o record sem comportamento adicional; a classificação é produzida pelo conciliador.

Dois resultados construídos separadamente, mas com os mesmos componentes, podem ser comparados por valor. Essa característica permite escrever uma lista esperada no teste e compará-la diretamente à resposta do conciliador.

Records não tornam automaticamente imutável qualquer objeto que carreguem. Neste modelo, porém, os componentes são **String**, um primitivo ou um enum, sem uma coleção mutável exposta. A documentação de records ajuda a entender o que a linguagem gera e o que continua sendo uma escolha do projeto.

5. Leia o serviço pelo fluxo principal

Antes de entrar nos detalhes, encontre a sequência de responsabilidades no método público. Ele prepara os índices, monta os resultados e devolve a resposta. A validação de um lançamento e a classificação de valores ficam em métodos menores.

Código completo: Conciliador.java

Os números à esquerda são referências de leitura; não fazem parte do arquivo Java ou XML.

```
 1  package br.com.wesley.conciliacao;
 2
 3  import java.util.ArrayList;
 4  import java.util.LinkedHashMap;
 5  import java.util.List;
 6  import java.util.Map;
 7
 8  public final class Conciliador {
 9
10      public List<Resultado> conciliar(
11              List<Lancamento> esperados, List<Lancamento> liquidados) {
12          Map<String, Long> esperadosPorReferencia = indexarEValidar(esperados,
     "esperados");
13          Map<String, Long> liquidadosPorReferencia = indexarEValidar(liquidados,
     "liquidados");
14
15          List<Resultado> resultados = new ArrayList<>();
16          for (Map.Entry<String, Long> esperado : esperadosPorReferencia.entrySet()) {
17              String referencia = esperado.getKey();
18              Status status = classificar(
19                      esperado.getValue(), liquidadosPorReferencia.get(referencia));
20              resultados.add(new Resultado(referencia, status));
21          }
22          for (String referencia : liquidadosPorReferencia.keySet()) {
23              if (!esperadosPorReferencia.containsKey(referencia)) {
24                  resultados.add(new Resultado(referencia, Status.NAO_PREVISTO));
25              }
26          }
27          return List.copyOf(resultados);
28      }
29
30      private Map<String, Long> indexarEValidar(List<Lancamento> lancamentos, String
     origem) {
31          if (lancamentos == null) {

32              throw new IllegalArgumentException(origem + ": a lista não pode ser
     nula.");
33          }
34          Map<String, Long> porReferencia = new LinkedHashMap<>();
35          int indice = 0;
36          for (Lancamento lancamento : lancamentos) {
37              validarLancamento(lancamento, origem, indice);
38              Long anterior = porReferencia.putIfAbsent(
```

```
39                      lancamento.referencia(), lancamento.valorCentavos());
40              if (anterior != null) {
41                  throw new IllegalArgumentException(
42                          origem + "[" + indice + "]: referência repetida: " +
    lancamento.referencia());
43              }
44              indice++;
45          }
46          return porReferencia;
47      }
48
49      private void validarLancamento(Lancamento lancamento, String origem, int indice) {
50          String local = origem + "[" + indice + "]";
51          if (lancamento == null) {
52              throw new IllegalArgumentException(local + ": o lançamento não pode ser
    nulo.");
53          }
54          String referencia = lancamento.referencia();
55          if (referencia == null || referencia.isBlank()) {
56              throw new IllegalArgumentException(local + ": a referência não pode ser
    nula ou em branco.");
57          }
58          if (lancamento.valorCentavos() <= 0) {
59              throw new IllegalArgumentException(local + ": o valor em centavos deve ser
     positivo.");
60          }

61      }
62
63      private Status classificar(long valorEsperado, Long valorLiquidado) {
64          if (valorLiquidado == null) {
65              return Status.NAO_LIQUIDADO;
66          }
67          return valorEsperado == valorLiquidado.longValue()
68                  ? Status.CONCILIADO
69                  : Status.VALOR_DIVERGENTE;
70      }
71  }
```

Cada linha, uma intenção

Linha	Instrução e explicação
1	`package br.com.wesley.conciliacao;` Declara o pacote do serviço, o mesmo dos records e do enum usados pela solução.
3	`import java.util.ArrayList;` Importa ArrayList, a implementação mutável usada somente para montar a resposta internamente.
4	`import java.util.LinkedHashMap;` Importa LinkedHashMap, que preserva a ordem de inserção ao construir os índices.
5	`import java.util.List;` Importa a interface List, usada nas entradas e no retorno do método público.

Linha	Instrução e explicação
6	`import java.util.Map;` Importa Map, o contrato dos índices por referência e de suas entradas.
8	`public final class Conciliador {` Declara o serviço público e final; final impede herança, enquanto a ausência de campos mutáveis mantém o trabalho por chamada.
10	`public List<Resultado> conciliar(` Inicia o método público conciliar e declara que seu retorno é uma lista de Resultado.
11	`List<Lancamento> esperados, List<Lancamento> liquidados) {` Recebe as duas listas de Lancamento e abre o corpo do método; a primeira representa expectativas e a segunda, liquidações.
12	`Map<String, Long> esperadosPorReferencia = indexarEValidar(esperados, "esperados");` Valida todos os esperados até o primeiro erro e, se forem aceitos, obtém o índice com sua ordem original.
13	`Map<String, Long> liquidadosPorReferencia = indexarEValidar(liquidados, "liquidados");` Faz a mesma preparação dos liquidados; a classificação só prossegue se os dois índices forem válidos.
15	`List<Resultado> resultados = new ArrayList<>();` Cria uma lista mutável local para acumular os resultados, depois de concluída a validação das entradas.
16	`for (Map.Entry<String, Long> esperado : esperadosPorReferencia.entrySet()) {` Percorre os pares de referência e valor dos esperados na ordem de inserção preservada pelo LinkedHashMap.
17	`String referencia = esperado.getKey();` Extrai a referência do par atual e a guarda em uma variável local.
18	`Status status = classificar(` Inicia a chamada ao método que decide o status deste pagamento esperado.
19	`esperado.getValue(), liquidadosPorReferencia.get(referencia));` Passa o valor esperado e consulta o valor liquidado pela mesma referência; a consulta pode devolver null.
20	`resultados.add(new Resultado(referencia, status));` Cria um Resultado com a referência atual e o status calculado e o acrescenta à resposta.
21	`}` Encerra o percurso dos esperados; cada referência desse lado já possui um resultado.

Linha	Instrução e explicação
22	`for (String referencia : liquidadosPorReferencia.keySet()) {` Percorre as referências dos liquidados na ordem em que chegaram.
23	`if (!esperadosPorReferencia.containsKey(referencia)) {` Verifica, com a negação de containsKey, se a referência não existe no índice dos esperados.
24	`resultados.add(new Resultado(referencia, Status.NAO_PREVISTO));` Acrescenta NAO_PREVISTO somente para um liquidado exclusivo, evitando repetir referências já classificadas.
25	`}` Fecha a condição que seleciona liquidados exclusivos.
26	`}` Encerra o percurso complementar dos liquidados.
27	`return List.copyOf(resultados);` Devolve uma lista não modificável com o conteúdo construído, sem expor o ArrayList de trabalho.
28	`}` Fecha o método público conciliar.
30	`private Map<String, Long> indexarEValidar(List<Lancamento> lancamentos, String origem) {` Declara o auxiliar privado que valida uma lista e devolve seu índice; origem identifica o lado nas mensagens de erro.
31	`if (lancamentos == null) {` Verifica se o próprio argumento da lista é null antes de tentar percorrê-lo.
32	`throw new IllegalArgumentException(origem + ": a lista não pode ser nula.");` Rejeita a lista nula com IllegalArgumentException, incluindo sua origem na mensagem.
33	`}` Fecha a guarda de nulidade da lista.
34	`Map<String, Long> porReferencia = new LinkedHashMap<>();` Cria um índice local com chaves String e valores Long, preservando a ordem de inserção.
35	`int indice = 0;` Inicia em zero o contador usado para identificar a posição de um lançamento inválido.
36	`for (Lancamento lancamento : lancamentos) {` Percorre cada lançamento da lista recebida sem remover ou substituir elementos.

Linha	Instrução e explicação
37	`validarLancamento(lancamento, origem, indice);` Valida o objeto, a referência e o valor antes de tentar inseri-los no mapa.
38	`Long anterior = porReferencia.putIfAbsent(` Inicia a inserção condicional e guarda o valor anterior associado à referência, se houver.
39	`lancamento.referencia(), lancamento.valorCentavos());` Usa a referência como chave e os centavos como valor; o long é convertido para Long ao entrar no mapa.
40	`if (anterior != null) {` Como o mapa não armazena valores nulos, um valor anterior não nulo identifica uma referência repetida.
41	`throw new IllegalArgumentException(` Inicia o lançamento da exceção que rejeita a duplicata, mesmo quando o valor repetido é igual.
42	`origem + "[" + indice + "]: referência repetida: " + lancamento.referencia());` Constrói a mensagem com origem, índice e referência repetida; fecha a construção da exceção.
43	`}` Fecha a condição de rejeição da duplicidade.
44	`indice++;` Avança o índice somente após o lançamento atual ser validado e inserido.
45	`}` Encerra o percurso da lista de lançamentos.
46	`return porReferencia;` Retorna o índice completo quando nenhum erro interrompeu a validação.
47	`}` Fecha o método indexarEValidar.
49	`private void validarLancamento(Lancamento lancamento, String origem, int indice) {` Declara o auxiliar privado sem retorno que valida um lançamento e recebe contexto para mensagens.
50	`String local = origem + "[" + indice + "]";` Monta a localização textual, como liquidados[1], a partir da origem e do índice iniciado em zero.
51	`if (lancamento == null) {` Testa se o elemento é null antes de chamar qualquer acessor do record.

Linha	Instrução e explicação
52	`throw new IllegalArgumentException(local + ": o lançamento não pode ser nulo.");` Rejeita um elemento nulo com a exceção de validação prevista no contrato.
53	`}` Fecha a guarda que protege o acesso ao lançamento.
54	`String referencia = lancamento.referencia();` Obtém a referência do record já confirmado como não nulo.
55	`if (referencia == null || referencia.isBlank()) {` Rejeita referência nula ou em branco; o curto-circuito de || evita chamar isBlank sobre null.
56	`throw new IllegalArgumentException(local + ": a referência não pode ser nula ou em branco.");` Lança a exceção com a localização do item cuja referência não foi aceita.
57	`}` Fecha a validação da referência sem aplicar trim nem alterar seu conteúdo.
58	`if (lancamento.valorCentavos() <= 0) {` Verifica se os centavos são zero ou negativos; ambos são proibidos.
59	`throw new IllegalArgumentException(local + ": o valor em centavos deve ser positivo.");` Rejeita o valor não positivo e informa a posição do lançamento.
60	`}` Fecha a validação do valor em centavos.
61	`}` Fecha validarLancamento; chegar aqui significa que as regras individuais foram atendidas.
63	`private Status classificar(long valorEsperado, Long valorLiquidado) {` Declara a classificação: o esperado sempre existe e é long; o liquidado é Long para permitir ausência representada por null.
64	`if (valorLiquidado == null) {` Verifica se a consulta ao índice dos liquidados não encontrou valor.
65	`return Status.NAO_LIQUIDADO;` Classifica como NAO_LIQUIDADO e encerra o método antes de qualquer conversão numérica do nulo.
66	`}` Fecha a condição de ausência; daqui em diante o valor liquidado é conhecido como não nulo.
67	`return valorEsperado == valorLiquidado.longValue()` Compara valores primitivos long; longValue explicita a extração numérica do Long liquidado.

Linha	Instrução e explicação
68	`? Status.CONCILIADO` No operador condicional, escolhe CONCILIADO quando a comparação de valores é verdadeira.
69	`: Status.VALOR_DIVERGENTE;` Escolhe VALOR_DIVERGENTE quando os dois valores existentes são diferentes.
70	`}` Fecha o método classificar.
71	`}` Fecha a classe Conciliador.

O nome **indexarEValidar** informa que a construção do mapa não aceita qualquer conteúdo. A função devolve um índice válido ou interrompe a chamada com uma exceção. Portanto, quando o primeiro laço de resultados começa, os dois lados já passaram pela validação.

Durante o percurso dos esperados, o programa sempre conhece a referência e o valor esperado. Ele consulta o mapa dos liquidados e entrega os dois valores a **classificar**. O método auxiliar responde apenas à pergunta sobre aquele esperado: está ausente, tem valor igual ou tem valor diferente?

O status **NAO_PREVISTO** não aparece nesse método porque ele representa uma situação que o percurso dos esperados não consegue encontrar: uma referência que existe exclusivamente no outro lado. Essa responsabilidade pertence ao segundo laço.

A condição **!esperadosPorReferencia.containsKey(referencia)** lê-se “o índice de esperados não contém esta referência”. O ponto de exclamação nega a condição. Quando ela é verdadeira, o serviço acrescenta um resultado com **NAO_PREVISTO**.

Não há necessidade de apagar do mapa os liquidados já classificados. Consultar a existência no índice de esperados é suficiente para evitar a repetição na saída. Isso mantém o algoritmo fácil de acompanhar: os índices são construídos e depois somente consultados.

Como putIfAbsent detecta a repetição

putIfAbsent insere a chave quando ela ainda não possui um valor associado e devolve o valor anterior conforme o contrato do mapa. Neste código, nenhum valor nulo é armazenado: todos vêm de um **long** previamente validado.

Por isso, retorno nulo significa primeira ocorrência; retorno não nulo significa que a referência já apareceu naquela lista. O valor anterior pode ser igual ao atual e, ainda assim, a entrada deve ser rejeitada.

Um **put** simples sobrescreveria o valor anterior e poderia esconder a duplicata. O mapa, sozinho, não “rejeita duplicados”: ele admite somente uma associação por chave, e a aplicação precisa

interpretar a tentativa de nova associação. Consulte o contrato de Map.putIfAbsent para não confundir os comportamentos.

6. Percorra o lote do enunciado, sem pular etapas

Vamos usar exatamente as referências e os valores do exemplo inicial. A disposição foi escolhida para mostrar todos os status e tornar a regra de ordem observável.

Posição	Esperados	Centavos	Liquidados	Centavos
0	pag-2	2000	pag-4	4000
1	pag-1	1000	pag-1	900
2	pag-3	3000	pag-2	2000

Primeiro: prepare os esperados

O serviço verifica se a lista é nula. Como não é, cria um **LinkedHashMap** vazio e começa no índice zero.

O lançamento **pag-2** existe, sua referência possui conteúdo e seu valor é positivo. A chave ainda não está no mapa, então ela é inserida. O índice avança. A mesma sequência acontece com **pag-1** e **pag-3**.

Ao terminar, a ordem do mapa é **pag-2** → **pag-1** → **pag-3**. Nenhum resultado foi criado até esse momento. O mapa é uma estrutura de consulta interna, não a resposta final.

Segundo: prepare os liquidados

A segunda chamada de **indexarEValidar** cria outro mapa. Suas inserções ocorrem na ordem **pag-4** → **pag-1** → **pag-2**.

Encontrar **pag-1** aqui não é duplicidade em relação aos esperados. A consulta de repetição acontece no mapa desta lista, que começou vazio. Como cada uma dessas chaves aparece uma única vez nos liquidados, a validação termina com sucesso.

Terceiro: classifique cada esperado

A primeira referência percorrida é **pag-2**. Seu valor esperado é 2000. A consulta no mapa dos liquidados encontra 2000, então o status é **CONCILIADO**. A lista de resultados recebe seu primeiro elemento.

Depois vem **pag-1**. O esperado é 1000, mas o liquidado é 900. A referência existe nos dois lados, portanto não é ausência; o problema é a diferença de valor. O segundo resultado é **VALOR_DIVERGENTE**.

Por fim, vem **pag-3**. O mapa dos liquidados não contém essa chave e a consulta devolve **null**. **classificar** retorna **NAO_LIQUIDADO** antes de tentar extrair qualquer valor numérico desse retorno.

Quarto: acrescente os liquidados exclusivos

O segundo laço percorre **pag-4**, **pag-1** e **pag-2**, nessa ordem. **pag-4** não existe nos esperados, então entra no fim da resposta com **NAO_PREVISTO**.

As outras duas referências já pertencem ao mapa dos esperados. Elas foram classificadas no primeiro laço e não devem gerar uma segunda linha.

Ordem final	Referência	Status	Razão
1	pag-2	CONCILIADO	2000 é igual a 2000
2	pag-1	VALOR_DIVERGENTE	1000 é diferente de 900
3	pag-3	NAO_LIQUIDADO	Não existe nos liquidados
4	pag-4	NAO_PREVISTO	Não existe nos esperados

Observe que **pag-4** chegou primeiro na lista do parceiro, mas aparece por último no resultado. A prioridade é preservar a sequência dos esperados. O índice do parceiro só determina a ordem relativa dos seus itens exclusivos.

Agora altere mentalmente a lista dos liquidados para **pag-5, pag-4, pag-2**. As referências exclusivas serão acrescentadas como **pag-5, pag-4**, depois de todos os esperados. Essa previsão é uma boa forma de verificar se você entendeu o mecanismo, e não apenas a saída decorada.

7. Validar o lote inteiro não é juntar todos os erros

A frase “validar o lote inteiro” pode gerar uma dúvida importante. Nesta solução, ela significa que a classificação só começa depois de todos os elementos dos dois lados terem sido aceitos. Se algo estiver errado, a chamada lança uma exceção e não devolve uma lista parcial.

O algoritmo trabalha em modo **fail-fast**: interrompe no primeiro erro encontrado. Ele não percorre o restante da entrada para produzir um relatório de todas as falhas. Reunir erros em uma coleção seria outro contrato, com outro tipo de retorno ou outra representação de rejeição.

Suponha que todos os esperados sejam válidos, mas o último liquidado tenha valor zero. O índice dos esperados já existe, e parte do índice dos liquidados pode ter sido construída. Isso é trabalho interno descartável. A lista de resultados ainda nem foi criada, porque sua declaração vem depois das duas chamadas de validação.

Também não existe resultado parcial escondido em um campo da classe, arquivo ou banco. O serviço não escreve em sistemas externos. Ao lançar a exceção, a chamada termina sem retornar normalmente.

A ordem das guardas evita erros acidentais. Primeiro verificamos a lista antes de percorrê-la. Depois verificamos cada lançamento antes de chamar seus acessores. Em seguida verificamos a referência antes de consultar seu conteúdo. Por fim avaliamos o valor e a duplicidade.

As mensagens incluem a origem e, quando disponível, o índice do elemento. **liquidados[1]** identifica a segunda posição, pois índices começam em zero. O contrato exige a categoria **IllegalArgumentException**; a mensagem fornece contexto para o diagnóstico.

Não é correto “pular os inválidos” e continuar. Isso ocultaria problemas de entrada e faria a resposta parecer completa. Também não é correto usar uma lista vazia como sinal de erro: listas vazias são uma entrada legítima e podem produzir legitimamente uma saída vazia.

Casos nulos merecem testes bem montados

List.of(null) já lança uma exceção durante a criação da lista. Assim, esse código não serve para demonstrar que o conciliador rejeita um elemento nulo: o serviço nem chega a receber a entrada.

Para esse teste, monte uma lista que aceite o elemento e só depois invoque o método sob teste:

```
List<Lancamento> entrada = new ArrayList<>();
entrada.add(null);

assertThrows(IllegalArgumentException.class,
        () -> conciliador.conciliar(entrada, List.of()));
```

O exemplo pressupõe os imports de **ArrayList**, **List** e das asserções do JUnit. Para testar a própria lista nula, passe **null** diretamente como um dos argumentos. Para testar uma referência

nula, construa **new Lancamento(null, 100L)**: o record passivo permite que o serviço receba esse caso.

8. Proteja as entradas e a resposta

As listas recebidas pertencem ao chamador. O conciliador pode percorrê-las, mas não deve ordenar, remover ou substituir seus elementos. A solução cria mapas próprios para consulta e uma lista própria para montar a resposta.

Isso permite receber tanto listas mutáveis quanto listas criadas por **List.of**. Se o algoritmo dependesse de remover um liquidado da lista original, ele poderia funcionar em um caso e falhar com **UnsupportedOperationException** no outro. Testar diferentes implementações de **List** ajuda a revelar esse acoplamento indevido.

A lista interna de resultados é um **ArrayList**, que aceita inclusões durante a montagem. No retorno, **List.copyOf(resultados)** fornece uma lista não modificável. O chamador não pode acrescentar, remover ou substituir elementos nessa resposta.

Não precisamos prometer que será criada uma nova identidade de objeto em toda situação: o compromisso relevante é o conteúdo retornado e sua impossibilidade de modificação. A API de List.copyOf descreve seu comportamento e as restrições sobre elementos nulos.

A lista não modificável e os resultados sem componentes mutáveis expostos formam uma resposta estável. O chamador pode guardar o retorno de uma chamada, executar outra conciliação e continuar consultando o primeiro resultado sem vê-lo ser reutilizado como área de trabalho.

Cada chamada começa do próprio começo

O conciliador não possui uma lista de resultados como campo, nem um mapa estático de referências já encontradas. Todos os dados auxiliares são criados dentro dos métodos.

Assim, uma chamada não herda pagamentos da anterior. Uma chamada inválida também não deixa um lote meio pronto armazenado na instância. Criar duas chamadas sobre o mesmo objeto deve produzir o mesmo comportamento que criar duas instâncias independentes para os mesmos dados.

Isso não autoriza modificar uma lista de entrada em outra thread enquanto a conciliação a percorre. Esse cenário de concorrência externa não foi contratado. O serviço não cria concorrência interna nem implementa um protocolo para capturar uma fotografia de listas que estejam sendo alteradas.

9. Explique a complexidade com precisão

Considere **n** como a quantidade de esperados e **m** como a quantidade de liquidados. Indexar os esperados percorre seus **n** elementos. Indexar os liquidados percorre seus **m** elementos. Os dois laços de classificação voltam a percorrer, respectivamente, até **n** e **m** referências.

A quantidade de resultados é, no máximo, **n + m**. Copiar a lista final também tem custo proporcional à quantidade produzida. Portanto, existem algumas passagens lineares; não existe uma busca completa da segunda lista dentro de cada passo da primeira.

Considerando operações de mapa com custo médio constante e referências de tamanho limitado, o tempo médio é **O(n + m)**. Os mapas, os resultados e a cópia final ocupam memória adicional **O(n + m)**.

O termo “médio” importa. Mapas baseados em hash possuem hipóteses de distribuição e custos internos; não precisamos prometer tempo constante absoluto para qualquer entrada. Duas chaves podem ter o mesmo hash sem serem iguais: o mapa também verifica igualdade. Colisão de hash não significa referência duplicada.

O comprimento do texto também custa. **isBlank()** examina caracteres; calcular hashes e comparar Strings pode exigir trabalho proporcional ao tamanho das referências. A expressão **O(n + m)** usa a convenção de tratar esse custo como limitado. Se referências gigantes fossem relevantes, a análise precisaria contabilizar também os caracteres processados.

Não use um cronômetro com poucos registros para “provar” essa complexidade. O raciocínio vem da estrutura do algoritmo. Medições de desempenho podem complementar a análise em um ambiente definido, mas não substituem a contagem das passagens e das buscas.

10. Use TDD como um roteiro de construção

TDD organiza o desenvolvimento em ciclos de teste que falha, implementação suficiente e melhoria do código. A sequência abaixo é uma proposta para você reconstruir a solução. Ela não afirma que os arquivos apresentados foram necessariamente produzidos nessa ordem temporal.

Vermelho: torne a ausência de comportamento visível

O enunciado fornece testes que utilizam **Conciliador**, **Lancamento**, **Resultado** e **Status**. Se esses tipos ainda não existem, o primeiro vermelho pode ser de compilação. Isso é diferente de um teste executado que encontrou uma classificação errada.

Crie os tipos e a assinatura pública necessários para o projeto compilar. Depois escolha uma regra pequena, como duas listas vazias ou um pagamento com valores iguais. Escreva a expectativa antes de preencher o comportamento correspondente.

Uma falha útil precisa ter uma razão compreensível. “Não compilou porque falta um tipo” orienta uma ação diferente de “esperava CONCILIADO, mas recebi VALOR_DIVERGENTE”.

Verde: atenda à regra sem ampliar o problema

Implemente o comportamento necessário e execute novamente. Acrescente casos para os quatro estados, para a ordem e para duplicatas com valor igual. A cada mudança, observe se uma expectativa já atendida deixou de funcionar.

Não introduza banco, filas ou mocks para fazer o exemplo parecer maior. As entradas são valores em memória e a saída pode ser comparada diretamente. Mockito não é necessário para testar um cálculo sem dependências externas.

A implementação deve continuar coerente com o contrato completo. Um retorno fixo que passa em um exemplo não encerra o trabalho; o próximo cenário deve expor a generalização que ainda falta.

Refatorar: melhorar mantendo as promessas

Com os testes passando, procure nomes que expliquem intenção e trechos com responsabilidades diferentes. Separar **validarLancamento**, **indexarEValidar** e **classificar** permite ler o fluxo público sem perder o detalhe quando ele é necessário.

Refatorar não é mudar a ordem exigida, normalizar referências ou ignorar entradas inválidas. Essas mudanças alterariam comportamento. Após uma refatoração, execute novamente a suíte para verificar se o contrato observável continua preservado.

Ao estudar, registre uma hipótese antes de alterar o código: “se eu trocar LinkedHashMap por uma estrutura sem ordem de inserção, qual teste deve acusar o problema?”. Esse hábito conecta decisão de implementação e evidência.

11. Entenda o que os testes realmente verificam

O arquivo **ConciliadorTest.java** mantém as expectativas iniciais do desafio. Ele verifica o lote com as quatro classificações e sua ordem, a duplicata com valor igual e a impossibilidade de acrescentar um resultado à lista devolvida.

assertEquals compara a lista esperada com a lista real. Para listas, a sequência dos elementos importa. Para os records usados aqui, a igualdade considera os componentes. A combinação permite verificar conteúdo e ordem de maneira legível.

assertThrows recebe a classe da exceção esperada e uma ação. O trecho `() -> conciliador.conciliar(...)` é uma lambda: uma pequena ação que o JUnit executará dentro da verificação. Se a chamada não lançar a exceção compatível, o teste falha.

Os testes complementares estão em **ConciliadorContratoTest.java**. Eles ampliam a observação por grupos:

Grupo	Erro que procura revelar
Listas vazias e um lado vazio	Confundir ausência de itens com entrada inválida
Ordem e referências exclusivas	Ordenar por chave, repetir resultados ou omitir liquidados
Valores pequenos, grandes e limites de long	Comparar objetos, converter para int ou perder precisão
Nulos, conteúdo em branco e valores não positivos	Validar somente um lado ou lançar exceção acidental
Duplicatas e referências exatas	Sobrescrever repetição, aplicar trim ou ignorar maiúsculas
Preservação e independência	Alterar entradas, permitir mudança da saída ou compartilhar estado

Há testes parametrizados que executam uma mesma intenção com diferentes argumentos. **ValueSource**, **EnumSource** e **MethodSource** fornecem esses conjuntos. O número de execuções pode ser maior que o número de métodos escritos; consulte o relatório atual em vez de deduzir a cobertura contando anotações.

Um teste com um erro no fim do lote verifica que a chamada inteira falha e que uma chamada posterior continua independente. Como a API não publica resultados intermediários, não é necessário inventar um mecanismo de captura de “meias respostas”. A estrutura do serviço explica por que elas não são devolvidas.

A documentação do JUnit 5.11.4 detalha asserções e testes parametrizados usados pelo projeto. O objetivo da suíte é demonstrar promessas do contrato, não apenas repetir linha por linha a implementação.

12. Maven: como o projeto é montado

O **pom.xml** descreve o projeto para o Maven. Ele informa sua identificação, a versão alvo do Java, a dependência de testes e os plugins usados nas etapas de compilação, teste e empacotamento.

Código completo: pom.xml

Os números à esquerda são referências de leitura; não fazem parte do arquivo Java ou XML.

```
 1  <?xml version="1.0" encoding="UTF-8"?>
 2  <project xmlns="http://maven.apache.org/POM/4.0.0"
 3           xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 4           xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
 5      <modelVersion>4.0.0</modelVersion>
 6      <groupId>br.com.wesley</groupId>
 7      <artifactId>conciliador-transacoes</artifactId>
 8      <version>1.0.0</version>
 9      <name>Conciliador de pagamentos</name>
10      <description>Desafio 03: conciliação de um lote, com Java 17 e JUnit
    5.</description>
11
12      <properties>
13          <maven.compiler.release>17</maven.compiler.release>
14          <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
15          <junit.version>5.11.4</junit.version>
16      </properties>
17
18      <dependencies>
19          <dependency>
20              <groupId>org.junit.jupiter</groupId>
21              <artifactId>junit-jupiter</artifactId>
22              <version>${junit.version}</version>
23              <scope>test</scope>
24          </dependency>
25      </dependencies>
26
27      <build>
28          <plugins>
29              <plugin>
30                  <groupId>org.apache.maven.plugins</groupId>
31                  <artifactId>maven-compiler-plugin</artifactId>
32                  <version>3.13.0</version>

33              </plugin>
34              <plugin>
35                  <groupId>org.apache.maven.plugins</groupId>
36                  <artifactId>maven-surefire-plugin</artifactId>
37                  <version>3.5.2</version>
38              </plugin>
39              <plugin>
40                  <groupId>org.apache.maven.plugins</groupId>
41                  <artifactId>maven-jar-plugin</artifactId>
42                  <version>3.4.2</version>
43                  <configuration>
```

```
44                      <archive>
45                          <manifest>
46
     <mainClass>br.com.wesley.conciliacao.ExemploConciliacao</mainClass>
47                          </manifest>
48                      </archive>
49                  </configuration>
50              </plugin>
51          </plugins>
52      </build>
53  </project>
```

Cada linha, uma intenção

Linha	Instrução e explicação
1	`<?xml version="1.0" encoding="UTF-8"?>` Declara um documento XML 1.0 codificado em UTF-8.
2	`<project xmlns="http://maven.apache.org/POM/4.0.0"` Abre o elemento project e informa o namespace do modelo POM usado pelo Maven.
3	`xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"` Declara o prefixo xsi, utilizado para indicar a localização do esquema XML.
4	`xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">` Associa o namespace do POM ao esquema oficial que descreve sua estrutura.
5	`<modelVersion>4.0.0</modelVersion>` Define a versão 4.0.0 do modelo POM; não é a versão instalada do executável Maven.
6	`<groupId>br.com.wesley</groupId>` Define o groupId, parte da identificação do projeto e de seu artefato.
7	`<artifactId>conciliador-transacoes</artifactId>` Define o artifactId conciliador-transacoes, utilizado no nome padrão do JAR.
8	`<version>1.0.0</version>` Define a versão 1.0.0 do projeto, também incluída no nome do artefato gerado.
9	`<name>Conciliador de pagamentos</name>` Fornece um nome legível para o projeto.
10	`<description>Desafio 03: conciliação de um lote, com Java 17 e JUnit 5.</description>` Descreve o objetivo educacional e as tecnologias escolhidas.
12	`<properties>` Abre o bloco de propriedades reutilizáveis na configuração.

Linha	Instrução e explicação
13	`<maven.compiler.release>17</maven.compiler.release>` Define Java 17 como release alvo da compilação; não instala nem escolhe sozinho o JDK do computador.
14	`<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>` Define UTF-8 para a leitura dos arquivos-fonte durante o build.
15	`<junit.version>5.11.4</junit.version>` Centraliza em uma propriedade a versão 5.11.4 do JUnit utilizada nos testes.
16	`</properties>` Fecha o bloco de propriedades.
18	`<dependencies>` Abre a lista de dependências do projeto.
19	`<dependency>` Inicia a declaração da dependência de testes JUnit Jupiter.
20	`<groupId>org.junit.jupiter</groupId>` Identifica o grupo org.junit.jupiter que publica o artefato.
21	`<artifactId>junit-jupiter</artifactId>` Seleciona o artefato junit-jupiter para o suporte aos testes JUnit 5.
22	`<version>${junit.version}</version>` Utiliza a propriedade junit.version em vez de repetir o número da versão.
23	`<scope>test</scope>` Limita essa dependência ao escopo de testes; o conciliador de produção não depende do JUnit.
24	`</dependency>` Fecha a declaração dessa dependência.
25	`</dependencies>` Fecha a lista de dependências.
27	`<build>` Abre as configurações do processo de build.
28	`<plugins>` Abre a lista de plugins utilizados nas etapas de construção.
29	`<plugin>` Inicia a configuração do plugin de compilação.

Linha	Instrução e explicação
30	`<groupId>org.apache.maven.plugins</groupId>` Identifica o grupo oficial dos plugins Maven.
31	`<artifactId>maven-compiler-plugin</artifactId>` Seleciona o Maven Compiler Plugin, responsável pela compilação Java.
32	`<version>3.13.0</version>` Fixa a versão 3.13.0 do plugin de compilação.
33	`</plugin>` Fecha a configuração do Compiler Plugin.
34	`<plugin>` Inicia a configuração do plugin que executa testes.
35	`<groupId>org.apache.maven.plugins</groupId>` Informa novamente o grupo dos plugins oficiais Maven.
36	`<artifactId>maven-surefire-plugin</artifactId>` Seleciona o Maven Surefire Plugin, usado para executar a suíte na fase de testes.
37	`<version>3.5.2</version>` Fixa a versão 3.5.2 do Surefire.
38	`</plugin>` Fecha a configuração do plugin de testes.
39	`<plugin>` Inicia a configuração do plugin responsável pelo JAR.
40	`<groupId>org.apache.maven.plugins</groupId>` Identifica o grupo oficial que publica o Jar Plugin.
41	`<artifactId>maven-jar-plugin</artifactId>` Seleciona o Maven Jar Plugin para empacotar as classes de produção.
42	`<version>3.4.2</version>` Fixa a versão 3.4.2 desse plugin.
43	`<configuration>` Abre a configuração específica do empacotamento.
44	`<archive>` Abre as opções do arquivo empacotado, incluindo seu manifesto.

Linha	Instrução e explicação
45	`<manifest>` Abre a configuração dos campos do manifesto do JAR.
46	`<mainClass>br.com.wesley.conciliacao.ExemploConciliacao</mainClass>` Indica o nome completo da classe com main, permitindo executar o exemplo com java -jar.
47	`</manifest>` Fecha a configuração do manifesto.
48	`</archive>` Fecha as opções do arquivo empacotado.
49	`</configuration>` Fecha a configuração específica do Jar Plugin.
50	`</plugin>` Fecha a declaração do plugin de empacotamento.
51	`</plugins>` Fecha a lista de plugins.
52	`</build>` Fecha o bloco de build.
53	`</project>` Fecha o documento de configuração do projeto Maven.

A propriedade **maven.compiler.release** está definida como **17**. Ela orienta o compilador a utilizar o conjunto de linguagem, formato de classe e APIs correspondente à versão alvo. Isso não instala um JDK no computador. O Maven ainda precisa ser executado com uma instalação adequada, conforme a documentação do Compiler Plugin.

JUnit aparece com escopo **test**. O programa de produção não precisa dessa biblioteca para comparar pagamentos. O Surefire executa os testes durante o build; o Jar Plugin registra no manifesto a classe que contém o ponto de entrada do exemplo.

O Maven Wrapper está no repositório para executar a versão de Maven configurada pelo projeto. Ele não é o Java e não substitui a instalação do JDK. Na primeira execução, pode precisar baixar o Maven e as dependências, portanto é necessário acesso aos repositórios correspondentes. Consulte o guia oficial do Wrapper.

Execute os comandos na raiz, a pasta que contém **pom.xml** e os scripts **mvnw** e **mvnw.cmd**.

macOS e Linux

```
java -version
javac -version
./mvnw -version
./mvnw clean verify
java -jar target/conciliador-transacoes-1.0.0.jar
```

Windows com PowerShell

```
java -version
javac -version
.\mvnw.cmd -version
.\mvnw.cmd clean verify
java -jar target/conciliador-transacoes-1.0.0.jar
```

No Prompt de Comando do Windows, use **mvnw.cmd clean verify** na mesma pasta. Se já houver uma instalação de Maven adequada, **mvn clean verify** também pode executar o projeto, mas o Wrapper ajuda a manter a versão prevista.

clean remove saídas anteriores do build. **verify** percorre as fases necessárias, incluindo compilação, testes e empacotamento, até a verificação. Se uma etapa falhar, a execução deve ser investigada antes de usar um artefato antigo como se representasse o código atual. O ciclo de vida do Maven explica essa sequência.

Após sucesso, o JAR esperado fica em **target/conciliador-transacoes-1.0.0.jar**. Os relatórios dos testes ficam em **target/surefire-reports**. Leia o resumo do build para confirmar o resultado atual; este livro não fixa uma quantidade de testes que poderia ficar desatualizada.

13. O exemplo executável e a fronteira da aplicação

O serviço de domínio não deve imprimir resultados. Para demonstrá-lo no terminal, o projeto possui uma classe separada que cria as entradas, chama a conciliação e mostra o retorno.

Código completo: ExemploConciliacao.java

Os números à esquerda são referências de leitura; não fazem parte do arquivo Java ou XML.

```
 1  package br.com.wesley.conciliacao;
 2
 3  import java.util.List;
 4
 5  public final class ExemploConciliacao {
 6      public static void main(String[] args) {
 7          List<Lancamento> esperados = List.of(
 8                  new Lancamento("pag-2", 2_000L),
 9                  new Lancamento("pag-1", 1_000L),
10                  new Lancamento("pag-3", 3_000L));
11          List<Lancamento> liquidados = List.of(
12                  new Lancamento("pag-4", 4_000L),
13                  new Lancamento("pag-1", 900L),
```

```
14                  new Lancamento("pag-2", 2_000L));
15
16          Conciliador conciliador = new Conciliador();
17          List<Resultado> resultados = conciliador.conciliar(esperados, liquidados);
18          for (Resultado resultado : resultados) {
19              System.out.println(resultado.referencia() + " -> " + resultado.status());
20          }
21      }
22  }
```

Cada linha, uma intenção

Linha	Instrução e explicação
1	`package br.com.wesley.conciliacao;` Declara o pacote da aplicação de demonstração, permitindo usar os tipos do domínio sem imports entre eles.
3	`import java.util.List;` Importa List para declarar as listas e utilizar a fábrica List.of.
5	`public final class ExemploConciliacao {` Declara a classe pública final que adapta a execução local ao serviço de domínio.
6	`public static void main(String[] args) {` Define o ponto de entrada main; ele é estático, não retorna valor e recebe os argumentos da linha de comando.
7	`List<Lancamento> esperados = List.of(` Inicia uma lista não modificável de lançamentos esperados usando List.of.
8	`new Lancamento("pag-2", 2_000L),` Adiciona pag-2 com 2000 centavos; o sublinhado é visual e L identifica um literal long.
9	`new Lancamento("pag-1", 1_000L),` Adiciona pag-1 com 1000 centavos na segunda posição dos esperados.
10	`new Lancamento("pag-3", 3_000L));` Adiciona pag-3 com 3000 centavos e fecha a construção da primeira lista.
11	`List<Lancamento> liquidados = List.of(` Inicia a lista não modificável de lançamentos informados como liquidados.
12	`new Lancamento("pag-4", 4_000L),` Coloca pag-4 primeiro no lado liquidado; ele será exclusivo desse lado.
13	`new Lancamento("pag-1", 900L),` Adiciona pag-1 com 900 centavos, preparando a divergência em relação aos 1000 esperados.

Linha	Instrução e explicação
14	`new Lancamento("pag-2", 2_000L));` Adiciona pag-2 com 2000 centavos e fecha a lista, preparando a correspondência de valores.
16	`Conciliador conciliador = new Conciliador();` Cria uma instância do conciliador, que não recebe dependências externas nem configuração.
17	`List<Resultado> resultados = conciliador.conciliar(esperados, liquidados);` Chama o serviço com as duas listas e guarda a resposta não modificável.
18	`for (Resultado resultado : resultados) {` Percorre os resultados na ordem devolvida pelo contrato.
19	`System.out.println(resultado.referencia() + " -> " + resultado.status());` Imprime referência, separador e status; essa saída de terminal pertence ao exemplo, não ao núcleo de domínio.
20	`}` Fecha o laço de impressão.
21	`}` Fecha o método main.
22	`}` Fecha a classe de demonstração.

O método **public static void main(String[] args)** é o ponto de entrada. **static** permite invocá-lo sem construir antes um objeto dessa classe; **void** informa que ele não devolve um valor ao chamador Java. **args** representa os argumentos recebidos pela linha de comando, embora o exemplo não precise utilizá-los.

A saída esperada do comando **java -jar** é:

```
pag-2 -> CONCILIADO
pag-1 -> VALOR_DIVERGENTE
pag-3 -> NAO_LIQUIDADO
pag-4 -> NAO_PREVISTO
```

A impressão pertence a **ExemploConciliacao**, não a **Conciliador**. Essa diferença permite que outro programa utilize a mesma regra e decida como apresentar o resultado: tela, arquivo ou resposta de uma operação.

O núcleo é um serviço de domínio puro no sentido adotado pelo exercício: recebe dados, aplica regras e retorna uma resposta ou lança uma exceção de validação, sem entrada e saída externas. Seus mapas locais são detalhes de cálculo. Não há motivo para criar uma interface de repositório quando nenhum repositório participa do problema.

Onde entraria a cloud? Duas frases

Um adaptador consumidor de mensagens receberia o lote de uma fila, converteria o payload em listas de Lancamento e chamaria Conciliador.conciliar. Desserialização, confirmação da mensagem, repetição e tratamento do transporte ficariam nesse adaptador, fora do conciliador, para preservar as regras de negócio independentes da infraestrutura.

14. Resolva erros sem alterar o contrato

O import aparece em vermelho

Abra na IDE a pasta que contém **pom.xml**, selecione um JDK 17 ou superior e importe ou recarregue o projeto Maven. Abrir apenas um arquivo solto pode deixar a IDE sem informação sobre a estrutura e as dependências.

Confira a grafia. O tipo é **java.util.List**; **Lancamento** e **Resultado** pertencem a **br.com.wesley.conciliacao**. Os testes também declaram esse pacote. JUnit utiliza nomes como **org.junit.jupiter.api.Test**, fornecidos pela dependência de teste.

Se o terminal compila e somente o editor reclama, verifique a configuração da IDE antes de reescrever o algoritmo. Se o terminal também falha, comece pela primeira mensagem relevante de compilação, pois erros posteriores podem ser consequência dela.

Java ou javac não foi encontrado

O projeto precisa de um JDK, que inclui o compilador. Confira a instalação e as configurações de **PATH** e **JAVA_HOME**. Depois abra um novo terminal e compare **java -version**, **javac -version** e a versão de Java exibida pelo Wrapper.

A mensagem **release version 17 not supported** normalmente indica que o Maven está usando um compilador antigo. Instalar outro JDK sem ajustar o ambiente usado pelo Maven pode deixar o problema igual.

O Wrapper não executa ou não baixa arquivos

No macOS/Linux, se aparecer **Permission denied**, confira se está na raiz e execute **chmod +x mvnw**. No Windows, utilize o script **mvnw.cmd**, com o prefixo *.* no PowerShell.

Falhas de download podem envolver conexão, proxy ou certificados do ambiente. Leia qual endereço e qual etapa falharam. Não contorne o problema desativando verificações de segurança nem alterando a regra de conciliação: o build ainda está preparando suas ferramentas.

O JAR não foi encontrado

Verifique se **clean verify** terminou com sucesso e se está executando o comando a partir da raiz. **Unable to access jarfile** pode significar caminho incorreto ou ausência de um empacotamento bem-sucedido.

Uma mensagem sobre ausência de **Main-Class** pede conferir o manifesto e o empacotamento do projeto. A configuração correspondente está no **pom.xml**; recompilar uma classe isolada não necessariamente recria o JAR executável.

A exceção é diferente da esperada

Se aparece **NullPointerException** antes de entrar no serviço, examine a construção da entrada. **List.of** não admite elementos nulos. Se a exceção acontece durante o acesso a uma referência, confira se a guarda de nulidade vem antes da chamada do método.

Se aparece **UnsupportedOperationException** ao modificar o retorno, a proteção solicitada está funcionando. O teste deve esperar essa exceção; não remova **List.copyOf** para silenciá-la.

Se a ordem está errada, investigue a estrutura usada e a sequência dos laços. Se duas referências com espaços parecem “injustamente” diferentes, releia o contrato: preservá-las exatamente é o comportamento correto deste exercício.

15. Exercícios para confirmar o entendimento

Tente responder antes de consultar a resolução. Escrever a previsão obriga você a distinguir o que imagina que o programa faça do que o contrato realmente pede.

Exercício 1 - quatro decisões

Esperados: **a = 100**, **b = 200**, **c = 300**. Liquidados: **d = 400**, **b = 250**, **a = 100**. Todos os valores estão em centavos. Qual é a saída, na ordem?

Resposta: a □ CONCILIADO, **b □ VALOR_DIVERGENTE**, **c □ NAO_LIQUIDADO**, **d □ NAO_PREVISTO**. A ordem inicial vem dos esperados; somente depois acrescentamos o liquidado exclusivo.

Exercício 2 - mesmo pagamento em dois lados

Esperados contém uma única entrada **pag-1 = 1000**. Liquidados também contém uma única entrada **pag-1 = 1000**. Isso é duplicidade?

Resposta: não. A repetição proibida ocorre dentro da mesma lista. Aqui há um pagamento presente uma vez em cada visão, que deve produzir um único resultado **CONCILIADO**.

Exercício 3 - repetir não é somar

Liquidados contém **pag-1 = 500** e **pag-1 = 500**, enquanto esperados contém **pag-1 = 1000**. Podemos somar os liquidados e conciliar?

Resposta: não. O lote é inválido por referência repetida nos liquidados. Somar transformaria o desafio em outro problema e esconderia a repetição que o contrato exige rejeitar.

Exercício 4 - espaços e letras

Esperados contém **"pag-1" = 100**. Liquidados contém **"PAG-1" = 100** e **" pag-1 " = 100**. Quantos resultados aparecem?

Resposta: três. O esperado recebe **NAO_LIQUIDADO**; as duas referências distintas do parceiro recebem **NAO_PREVISTO**, na ordem apresentada. Não há trim nem conversão de maiúsculas e minúsculas.

Exercício 5 - erro no final

Uma lista de liquidados começa com vários itens válidos e termina com um lançamento de valor zero. O serviço deve devolver os resultados dos itens anteriores?

Resposta: não. Ele interrompe a chamada com **IllegalArgumentException**. Os dois índices precisam estar validados antes de começar a lista de resultados; nenhum retorno parcial é disponibilizado.

Exercício 6 - o nulo apareceu cedo demais

Um teste escreve **List.of((Lancamento) null)** fora de **assertThrows** e falha antes de chamar o conciliador. Qual é o problema?

Resposta: a fábrica da lista rejeita o elemento nulo. Construa uma lista que permita esse elemento e coloque somente a chamada ao serviço na ação verificada. Assim o teste observa a responsabilidade correta.

Exercício 7 - uma troca aparentemente inocente

Você substitui **LinkedHashMap** por **TreeMap** porque ele também permite buscar por chave. O que muda?

Resposta: o percurso passa a seguir a ordenação das chaves. Referências recebidas como **z, a, m** poderiam sair como **a, m, z**, contrariando o contrato. Ser um mapa não basta para preservar a ordem necessária.

Exercício 8 - uma variável fora do método

Para evitar alocações, você transforma a lista de resultados em um campo da classe e a reutiliza a cada chamada. Que risco introduz?

Resposta: o objeto passa a carregar estado entre chamadas. Dependendo de como esse campo for exposto ou limpo, resultados podem acumular, respostas anteriores podem ser afetadas e falhas podem deixar resíduos. A solução atual prefere variáveis locais e chamadas independentes.

Exercício 9 - comparar Long

Dois objetos **Long** representam 10.000 centavos. Comparar suas referências com == demonstra necessariamente que os valores são iguais?

Resposta: não. Identidade de objeto e igualdade numérica são perguntas diferentes. No conciliador, a presença é verificada antes da conversão para primitivo, e a comparação ocorre entre valores **long**.

Exercício 10 - mude uma hipótese

O negócio agora quer aceitar valores zero, coletar todos os erros e normalizar referências. Basta acrescentar três condições?

Resposta: primeiro é preciso reformular o contrato. Aceitar zero muda a validação; coletar erros exige definir como eles serão devolvidos; normalizar chaves pode criar colisões entre referências antes distintas. Atualize os exemplos e os testes antes de implementar essas novas decisões.

16. Explique a solução com suas próprias palavras

Você pode apresentar a implementação assim:

“Valido cada lista e construo um índice por referência com LinkedHashMap, preservando a ordem de inserção e rejeitando duplicatas no próprio lado. Depois percorro os esperados para comparar os valores ou identificar ausência, acrescento os liquidados exclusivos e devolvo uma lista não modificável.”

Para justificar o custo, acrescente: “Cada lista é percorrida um número fixo de vezes, com consultas médias por chave; por isso o tempo médio e a memória adicional são O(n + m), considerando o tamanho das referências limitado.”

Antes de encerrar, confira se consegue explicar por que uma chave presente nos dois lados é válida, por que uma referência em branco é diferente de uma referência com espaços ao redor de texto e por que **Long** aparece onde pode existir ausência. Essas três respostas mostram compreensão do contrato, dos dados e da linguagem.

Referências para continuar estudando

As fontes abaixo documentam os recursos usados. As decisões do exercício, como rejeitar duplicatas e não aplicar trim, vêm do enunciado.

- Java 17 - Records
- Java 17 - Map e putIfAbsent
- Java 17 - LinkedHashMap
- Java 17 - List e fábricas não modificáveis
- Java 17 - String e isBlank
- Java 17 - Long
- Java 17 - Avaliação condicional de expressões
- JUnit 5.11.4 - Guia de uso
- Apache Maven - Wrapper
- Apache Maven - Ciclo de vida do build
- Maven Compiler Plugin - Configuração de release

As versões do projeto estão fixadas para tornar o exercício reproduzível. Elas não representam uma afirmação de que sejam as versões mais recentes disponíveis.